

Préparation à la soutenance — PlatformServerless

Préparation à la soutenance — PlatformServerless

Questions techniques du jury et du chef de projet, avec réponses argumentées

Adel Bettaieb — PFE ESPRIT — NextStep IT

Comment utiliser ce document

Chaque question est suivie de trois blocs :

- **Réponse** — ce qu'il faut dire, structuré pour être dit à l'oral.
- **Preuve dans le code** — la classe, l'annotation ou l'endpoint exact à citer si on te demande « où ça, dans ton code ? ». C'est ce qui fait la différence entre un candidat qui récite et un candidat qui maîtrise.
- **Piège / à éviter** — l'erreur classique sur cette question précise.

Les questions marquées **[PIÈGE]** portent sur des faiblesses réelles de ton projet. Le jury les trouvera. Mieux vaut les avoir préparées que les découvrir en direct.

Règle générale pour toute la soutenance : **assume tes choix, ne les invente pas**. Un « je n'ai pas implémenté ça, voici pourquoi et ce que je ferais » vaut toujours mieux qu'une réponse floue qui s'effondre à la deuxième relance.

1. Architecture générale et positionnement

Q1.1 — En une phrase, qu'est-ce que ta plateforme ?

Réponse. C'est une plateforme PaaS multi-tenant qui permet à un client de déployer une application conteneurisée à partir d'une simple image Docker, sans connaître Kubernetes, avec une exécution serverless : l'application descend à zéro instance quand elle n'est pas sollicitée, remonte automatiquement à la première requête, et n'est facturée que sur sa consommation réelle.

Piège. Ne commence pas par « c'est une application Spring Boot avec React ». Ça décrit ta stack, pas ton produit. Le jury veut d'abord comprendre la valeur, la technique vient après.

Q1.2 — Pourquoi Knative plutôt que de gérer toi-même les Deployments Kubernetes ?

Réponse. Trois raisons. D'abord le scale-to-zero : Kubernetes natif ne sait pas descendre un Deployment à zéro réplica puis le réveiller sur une requête HTTP entrante — il faudrait réimplémenter l'Activator, la mise en file d'attente et le réveil, ce qui est un projet à part entière. Ensuite le modèle de révisions : chaque modification d'un Service Knative crée une Revision figée, ce qui donne l'historique et le rollback gratuitement. Enfin le routage : Knative génère l'URL et la Route, on n'a pas à gérer manuellement un Ingress par application.

Preuve dans le code. `KnativeService.deploy()` ne crée qu'une seule ressource, un `Service` de l'API `serving.knative.dev/v1`. La Revision, le Deployment et le Pod en sont dérivés automatiquement par Knative — c'est exactement ce que montre la figure de la chaîne d'intégration backend→Knative. Le rollback (`rollbackToRevision()`) se contente de patcher `spec.traffic` à 100 % sur la révision cible.

Piège. Si on te demande « et le scale-to-zero, tu l'as codé ? », la réponse est **non, c'est natif à Knative**. C'est une force, pas une faiblesse : réimplémenter ça aurait été une mauvaise décision d'ingénierie. Dis-le ainsi.

Q1.3 — Pourquoi un cluster on-premise et pas EKS, GKE ou AKS ?

Réponse. Deux raisons, une métier et une budgétaire. NextStep IT est un intégrateur dont l'offre IaaS repose précisément sur la maîtrise de sa propre infrastructure : construire la plateforme sur du cloud managé aurait été contradictoire avec le positionnement de l'entreprise. Et aucun budget cloud récurrent n'était prévu pour un PFE.

La conséquence technique directe, c'est MetalLB : sur un cluster managé, un Service de type LoadBalancer reçoit automatiquement une IP publique. Sur du bare-metal, ce contrôleur n'existe pas, le Service resterait indéfiniment en `<pending>`. MetalLB joue ce rôle en attribuant des IP depuis une plage réservée du réseau local.

Piège. Ne présente pas le on-premise comme un choix par défaut ou une contrainte subie. C'est une décision d'architecture cohérente avec l'entreprise d'accueil — et tu peux citer son coût : MetalLB, Kourier et le provisionnement manuel sont exactement ce qu'un cluster managé t'aurait offert clé en main.

Q1.4 — Quelle est la différence entre ton backend et un opérateur Kubernetes ?

Réponse. Un opérateur tourne dans le cluster, surveille des ressources personnalisées et réconcilie en boucle l'état réel vers l'état désiré. Mon backend n'est pas un opérateur : c'est un client de l'API Kubernetes. Il traduit une demande métier — « ce client veut déployer cette image » — en manifestes Kubernetes, puis délègue la

réconciliation aux vrais opérateurs (Knative Serving pour les Services, Strimzi pour Kafka).

Il y a cependant un composant qui s'en rapproche : `KnativeWatcher`, qui ouvre un watch permanent sur les ressources Knative de tous les namespaces et met à jour le statut en base à chaque événement. C'est un mécanisme d'observation, pas de réconciliation.

Preuve dans le code. `KnativeWatcher` utilise `kubernetesClient...watch(new Watcher<>() {...})`, avec gestion de la reconnexion dans `onClose()`. Il ne modifie jamais le cluster, il ne fait que lire pour synchroniser la base.

2. Kubernetes, réseau et isolation

Q2.1 — Comment garantis-tu qu'un client ne peut pas accéder aux applications d'un autre client ?

Réponse. L'isolation repose sur trois couches superposées, et c'est important qu'elles soient distinctes :

1. **Isolation logique par namespace** — chaque tenant obtient son propre namespace Kubernetes, nommé `user-<username>`, créé dynamiquement au premier déploiement.
2. **Isolation réseau par NetworkPolicy** — une politique `default-deny` bloque tout trafic non explicitement autorisé, en entrée comme en sortie. Elle n'ouvre que les flux nécessaires : composants système Knative et Kourier, résolution DNS, et le bus Kafka partagé. Le trafic direct entre deux namespaces de tenants est refusé.
3. **Isolation applicative par contrôle de propriété** — même si un client devinait l'identifiant d'une application d'un autre client, le backend vérifie systématiquement la propriété avant toute opération.

Preuve dans le code. Pour la couche 3, la méthode `AppService.requireOwned(appId, userId)` lève une `UnauthorizedException` si `app.getUserId()` ne correspond pas à l'utilisateur effectif. Elle est appelée dans **toutes** les méthodes qui touchent une application : `getApp`, `updateApp`, `redeploy`, `deleteApp`, `listRevisions`, `rollback`. C'est la défense contre l'IDOR.

Pour la couche 1, `UserContextService.buildNamespace()` construit le namespace, et la fonction est volontairement `static` pour être réutilisable par `QuotaService`.

Piège. Si tu ne cites que le namespace, le jury enchaînera : « et si je change l'ID dans l'URL ? ». Cite les trois couches d'emblée, en insistant sur `requireOwned` — c'est la réponse à cette question-là.

Q2.2 — Pourquoi Cilium et pas Calico ou Flannel ?

Réponse. Flannel est éliminé d'emblée : il ne supporte pas nativement les NetworkPolicy, or l'isolation multi-tenant est une exigence centrale du projet, pas une option. Restaient Calico et Cilium, qui supportent tous deux les NetworkPolicy de façon

comparable. Cilium a été retenu pour son architecture eBPF : le filtrage se fait dans le noyau Linux sans passer par les chaînes iptables, ce qui monte mieux en charge quand le nombre de règles augmente, et ouvre la porte à de l'observabilité réseau avancée (Hubble) si la plateforme évoluait.

Piège. Ne prétends pas avoir fait un benchmark de performance entre Calico et Cilium si tu ne l'as pas fait. La réponse honnête est : « les deux répondaient au besoin fonctionnel, j'ai choisi Cilium pour son potentiel d'évolution ». C'est un critère de choix parfaitement valide, et l'assumer est plus solide qu'inventer des mesures.

Q2.3 — Explique-moi le chemin complet d'une requête HTTP externe jusqu'au conteneur du client.

Réponse. Sept étapes :

1. Le client externe résout le nom d'hôte `<app>.<namespace>.<ip-cluster>.sslip.io`. `sslip.io` renvoie l'IP encodée dans le nom lui-même, sans zone DNS à administrer.
2. La requête atteint l'IP externe attribuée par MetalLB au service d'entrée.
3. **Kourier**, la passerelle d'entrée de Knative, reçoit la requête et lit le nom d'hôte.
4. Il identifie la Route Knative correspondante, donc l'application cible.
5. Si l'application a zéro instance, la Route pointe vers l'**Activator** : la requête est mise en file d'attente, et l'Activator signale la demande à l'**Autoscaler** (KPA).
6. L'Autoscaler passe le Deployment de la Revision de 0 à 1 réplica. Le Pod démarre.
7. La requête en attente est transmise au Pod, qui contient le conteneur client et le sidecar `queue-proxy` — ce dernier remonte les métriques de concurrence à l'Autoscaler.

Si l'application a déjà une instance active, on saute les étapes 5 et 6, la requête va directement au Pod.

Piège. Le `queue-proxy` est le détail qui distingue une réponse apprise d'une réponse comprise. C'est lui qui mesure la concurrence réelle et permet à l'Autoscaler de décider. Cite-le.

Q2.4 — Ta convention de nommage `sslip.io`, c'est viable en production ?

Réponse. Non, et c'est assumé. `sslip.io` est un service public qui résout n'importe quel nom contenant une IP vers cette IP. C'est parfait pour un cluster de développement on-premise sans infrastructure DNS : zéro configuration. En production, il faudrait un domaine dédié avec une vraie zone DNS, un certificat TLS wildcard, et idéalement un `external-dns` qui crée les enregistrements automatiquement à chaque déploiement.

Le coût actuel est double : des URL peu lisibles, et une dépendance à un service tiers pour la résolution.

Piège. Si tu défends sslip.io comme une solution de production, tu perds la crédibilité de tout le reste. Reconnais la limite, propose la solution — c’est exactement ce qu’on attend d’un ingénieur.

3. Sécurité et authentification

Q3.1 — Comment fonctionne l’authentification de bout en bout ?

Réponse. Keycloak est le fournisseur d’identité unique. Le backend ne gère **ni mot de passe, ni session** : c’est un serveur de ressources OAuth2 (`oauth2ResourceServer`), configuré en mode `STATELESS`. Il valide la signature du jeton JWT émis par Keycloak, en extrait le rôle applicatif via un convertisseur dédié, et autorise ou refuse.

Le portail React échange les identifiants contre un jeton auprès de Keycloak, puis joint ce jeton en en-tête `Authorization: Bearer` à chaque appel. Un rafraîchissement automatique est en place via le `refresh_token`.

Preuve dans le code. `SecurityConfig.filterChain()` : `.sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))`
et `.oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> jwt.jwtAuthenticationConverter(keycloakJwtAuthConverter)))`. La conversion du rôle Keycloak vers le rôle Spring est isolée dans `KeycloakJwtAuthConverter`.

Q3.2 — [PIÈGE] Tu utilises quel flow OAuth2 exactement ?

C’est **la** question technique la plus dangereuse de ta soutenance. Un jury qui connaît OAuth2 la posera.

Réponse. Le portail utilise le flow **Resource Owner Password Credentials (ROPC)** : il envoie directement le login et le mot de passe à l’endpoint `token` de Keycloak avec `grant_type=password`, et reçoit le jeton en retour.

Il faut assumer que **ce n’est pas le flow recommandé**. ROPC est déprécié dans OAuth 2.1 pour une raison précise : l’application cliente manipule le mot de passe en clair, ce qui est acceptable uniquement quand le client et le serveur d’autorisation appartiennent à la même organisation — ce qui est le cas ici — mais empêche structurellement le SSO, la fédération d’identité et le MFA porté par Keycloak.

Le flow correct serait **Authorization Code + PKCE** : redirection vers la page de login Keycloak, retour avec un code d’autorisation, échange du code contre un jeton. Le mot de passe ne transite jamais par l’application.

Preuve dans le code. `AuthContext.jsx` fait un `axios.post(TOKEN_URL, new URLSearchParams({ grant_type: 'password', client_id, username, password }))`. Détail à connaître : un fichier `web-portal/src/auth/keycloak.js` utilisant le vrai adaptateur `keycloak-js` existe dans le dépôt mais **n’est importé nulle part** — c’est du code mort,

vestige d'une première approche. Si le jury le trouve, mieux vaut que tu l'aies signalé toi-même.

Stratégie de défense. Trois temps : (1) nomme le flow correctement, ROPC, sans hésiter ; (2) explique pourquoi il est déconseillé, en montrant que tu connais l'alternative PKCE ; (3) explique le contexte qui l'a rendu acceptable ici — client et IdP dans la même organisation, périmètre de PFE, pas de fédération externe requise — et dis que la migration vers PKCE est identifiée dans les perspectives. Ce risque est d'ailleurs documenté dans le chapitre sécurité de ton mémoire, ce qui prouve que tu l'avais vu avant le jury.

Piège absolu. Ne dis jamais « j'utilise OAuth2 avec Keycloak » en espérant que ça passe. Si le jury creuse et découvre que tu ne savais pas quel flow tu utilisais, c'est bien plus grave que le choix de ROPC lui-même.

Q3.3 — Comment gères-tu les permissions fines d'un membre d'équipe ?

Réponse. Le modèle a deux niveaux. D'abord trois rôles : `ADMIN` (exploitant de la plateforme), `CLIENT_ADMIN` (client titulaire d'un compte) et `MEMBER` (collaborateur invité par un `CLIENT_ADMIN`). Ensuite, pour les `MEMBER` uniquement, un jeu de permissions granulaires accordées individuellement : `DEPLOY_APP`, `DELETE_APP`, `MANAGE_KAFKA`, `MANAGE_EVENTING`, `VIEW_LOGS`, `VIEW_MONITORING`, `VIEW BILLING`, `EXPORT BILLING`.

La règle d'évaluation est simple : `ADMIN` et `CLIENT_ADMIN` sont toujours autorisés ; seul un `MEMBER` est vérifié contre la liste de permissions que son `CLIENT_ADMIN` lui a explicitement accordées.

Preuve dans le code. `PermissionService.has(username, permission)` :

```
if (!"MEMBER".equals(u.getRole())) return true; // ADMIN / CLIENT_ADMIN
return u.getPermissions() != null && u.getPermissions().contains(permission);
```

L'appel se fait par annotation Spring, directement sur l'endpoint : `@PreAuthorize("@permissionService.has(authentication.name, 'DEPLOY_APP'))` sur `AppController.createApp()`.

Question de relance probable : « et un `MEMBER`, il déploie dans quel namespace ? ». Réponse : dans celui de son `CLIENT_ADMIN`, pas dans un namespace propre. C'est `UserService.resolve()` qui gère ça : si `user.getOwnerId() != null`, la méthode retourne l'identifiant **et** le namespace du propriétaire. Un `MEMBER` n'a donc pas d'espace à lui — il travaille dans l'espace de l'équipe.

Q3.4 — Comment un CLIENT_ADMIN ajoute-t-il un membre ? Il y a un e-mail d'invitation ?

Réponse. Non, et c'est un choix à assumer. Le CLIENT_ADMIN saisit lui-même le nom d'utilisateur, l'e-mail et le mot de passe du futur membre. Le backend crée alors deux choses : un compte dans Keycloak avec ce mot de passe marqué comme **permanent** (pas de réinitialisation forcée à la première connexion), et un utilisateur local avec le rôle `MEMBER` et un `ownerId` pointant vers le CLIENT_ADMIN. Le membre peut se connecter immédiatement, avec des identifiants qui lui sont transmis hors du système.

Preuve dans le code. `TeamService.addMember()` : vérification d'unicité sur l'e-mail et le `username`, puis `keycloakAdminService.createUser(...)`, puis `User.builder().role("MEMBER").ownerId(ownerId).build()`. Les quatre endpoints de `TeamController` sont protégés par `@PreAuthorize("hasRole('CLIENT_ADMIN')")`.

Limite à reconnaître si on te pousse. Un vrai flux d'invitation par e-mail avec jeton à usage unique serait plus sûr : il éviterait que le CLIENT_ADMIN connaisse le mot de passe de son collaborateur. Autre point : aucun rôle de royaume Keycloak n'est assigné au membre — la délégation n'existe que dans la base locale. C'est fonctionnel, mais ça signifie que Keycloak seul ne connaît pas la hiérarchie d'équipe.

Q3.5 — Que se passe-t-il si quelqu'un s'inscrit tout seul via le formulaire public ?

Réponse. `POST /api/auth/register` est effectivement public. Le compte est créé dans Keycloak **sans rôle de royaume assigné**, puis le filtre `UserSyncFilter` crée l'utilisateur local avec le rôle `MEMBER` par défaut et **sans `ownerId`**.

C'est une situation volontairement inerte : un tel compte ne peut rien faire. Il n'est rattaché à aucun CLIENT_ADMIN, donc `UserContextService.resolve()` le traite comme son propre propriétaire, mais comme il a le rôle `MEMBER`, `PermissionService.has()` le vérifie contre une liste de permissions vide. Toute action protégée est refusée tant qu'un ADMIN ne lui attribue pas un rôle actif via `PATCH /api/users/{id}/role`.

Piège. Si on te demande « donc n'importe qui peut créer un compte sur ta plateforme ? », la réponse est oui, mais sans aucun privilège. C'est une file d'attente de validation, pas une faille. Formule-le dans ce sens.

Q3.6 — Comment sont stockées les clés d'API ?

Réponse. La clé est générée avec `SecureRandom`, puis **seul son empreinte SHA-256 est persistée** — la valeur en clair n'est retournée qu'une seule fois, au moment de la création, et n'est jamais récupérable ensuite. Un préfixe de 12 caractères est stocké en clair séparément, uniquement pour permettre à l'utilisateur d'identifier visuellement sa clé dans la liste.

À la vérification, `ApiKeyFilter` recalcule le SHA-256 de la clé reçue et cherche la correspondance parmi les clés actives.

Preuve dans le code. `ApiKeyService` : `SecureRandom RANDOM`, `String hash = sha256(rawKey)`, `String prefix = rawKey.substring(0, 12)`, et le lookup par `repository.findByKeyHashAndActiveTrue(hash)`.

Relance possible : « pourquoi SHA-256 et pas BCrypt, comme pour les mots de passe ? ». Bonne question, et il y a une vraie réponse : BCrypt est volontairement lent, ce qui est souhaitable contre le bruteforce d'un mot de passe humain à faible entropie. Une clé d'API générée par `SecureRandom` a une entropie très élevée, le bruteforce n'est pas la menace, et elle est vérifiée à chaque requête d'un pipeline CI/CD — la lenteur de BCrypt serait ici un coût sans bénéfice. SHA-256 est le choix standard pour ce cas d'usage.

Q3.7 — [PIÈGE] Tu m'as dit que les `NetworkPolicy` isolent les tenants. Elles sont appliquées partout ?

Réponse honnête. Non, et c'est documenté dans mon mémoire comme une limite constatée. Lors de la vérification finale, seul le namespace `user-user` portait effectivement la règle `tenant-default-isolation`. Les trois autres namespaces tenants n'en avaient aucune.

L'explication tient à l'historique : la création automatique de la `NetworkPolicy` à chaque nouveau namespace a été introduite par le backend en cours de projet. Les namespaces créés **avant** cette automatisation n'ont pas reçu la règle rétroactivement, et aucun script de rattrapage n'a été exécuté.

Stratégie de défense. Ne minimise pas : c'est un écart réel entre le modèle de sécurité conçu et l'état déployé. Ce qu'il faut montrer, c'est (1) que tu l'as détecté toi-même par une vérification systématique, pas par hasard, (2) que tu l'as documenté au lieu de le cacher, et (3) que tu sais comment le corriger : un job de réconciliation au démarrage du backend qui parcourt tous les namespaces `user-*` et applique la politique manquante — ce qui rapprocherait d'ailleurs le backend d'un vrai opérateur.

Piège. Si tu affirmes que l'isolation réseau est totale et que le jury a lu ta capture `kubectl get networkpolicy`, la contradiction est visible à l'écran. Prends les devants.

4. Kafka, Eventing et messagerie

Q4.1 — Pourquoi `AdminClient` pour Kafka alors que tu utilises Fabric8 partout ailleurs ?

Réponse. Parce que ce sont deux plans différents. Fabric8 est un client de l'**API Kubernetes** : il sert à manipuler des ressources Kubernetes, y compris les CRD de Knative Serving et Eventing. Créer un topic Kafka n'est pas une opération Kubernetes,

c'est une opération du **protocole Kafka lui-même** — elle s'adresse aux brokers, pas à l'API server.

Strimzi propose bien une CRD `KafkaTopic` qui permettrait de passer par Kubernetes, mais elle ajoute une indirection : on crée un objet Kubernetes, l'opérateur Strimzi le voit, et crée le vrai topic. Avec `AdminClient`, on parle directement aux brokers, la création est synchrone et l'erreur remonte immédiatement.

Preuve dans le code. La connexion se fait vers le service de bootstrap `my-cluster-kafka-bootstrap` sur le port 9092, sans passer par l'API Kubernetes. C'est précisément ce que la figure de la chaîne d'intégration `AdminClient` illustre, avec l'encart « l'API Kubernetes n'intervient pas dans ce flux ».

Attention à un piège de nommage. Il existe une classe `KafkaTopic` dans ton backend. Ce **n'est pas** la CRD Strimzi du même nom : c'est une entité JPA qui persiste les métadonnées du topic en base (propriétaire, date, nom). Si le jury voit ce nom, il peut croire que tu utilises la CRD. Devance la confusion.

Q4.2 — Décris-moi précisément ce qui se passe entre la publication d'un message Kafka et l'exécution du code du client.

Réponse. Onze étapes, c'est le cœur technique du projet :

1. Un producteur publie un message sur un topic Kafka.
2. La **KafkaSource** consomme ce message depuis le topic.
3. Elle l'encapsule au format **CloudEvent** et l'envoie en HTTP au Broker.
4. Le **Broker** Knative Eventing reçoit l'événement.
5. Le **Trigger**, qui filtre sur le type d'événement, identifie l'application cible.
6. Si l'application est à zéro instance, l'**Activator** intercepte et met l'événement en file.
7. L'**Autoscaler** (KPA) est notifié de la demande.
8. Il passe le Deployment de la Revision de 0 à 1 réplica.
9. Le **Pod** démarre, reçoit l'événement, le traite, répond HTTP 200.
10. Après une période d'inactivité, l'Autoscaler redescend à 0 réplica.
11. Le cycle recommence à la publication suivante.

Preuve dans le code. `EventingService.createTrigger()` et `createKafkaSource()`. Un point important : `ensureBrokerExists(namespace)` est appelé avant la création de la `KafkaSource`, car **Knative exige qu'un Trigger et son Broker soient dans le même namespace**. C'est une contrainte réelle rencontrée pendant le développement, et c'est un excellent détail à citer — il montre que tu as buté dessus et compris pourquoi.

Preuves d'exploitation : les trois captures `kubectl get broker`, `kubectl get trigger` et `kubectl get kafkasource` dans le mémoire.

Q4.3 — Le déploiement d’une application peut créer automatiquement un déclencheur Kafka ?

Réponse. Oui. Si le formulaire de déploiement renseigne un topic Kafka, le déploiement asynchrone crée dans la foulée la `KafkaSource` et le `Trigger`, sans étape manuelle. Le nom de la source est dérivé du service (`<serviceName>-source`) et le filtre d’événement vaut `order.created` par défaut si l’utilisateur n’en précise pas.

Preuve dans le code. Dans `AppDeploymentAsyncRunner.triggerDeploy()`, après le déploiement réussi :

```
if (Boolean.TRUE.equals(req.getKafkaEnabled()) && req.getKafkaTopicId() != null) {
    var source = eventingService.createKafkaSource(...);
    eventingService.createTrigger(app.getUserId(), source.getId(), filter, url);
}
```

Le câblage est journalisé avec le type `KAFKA_WIRED`, ce qui le rend traçable dans les journaux de déploiement.

Q4.4 — [PIÈGE] Ton cluster Kafka a un seul broker et utilise ZooKeeper. C’est un choix ?

Réponse. C’est un dimensionnement d’environnement de développement, assumé comme tel. La ressource `Kafka` déclare un broker et une instance `ZooKeeper`, chacun sur un volume persistant de 1 Gio en classe `local-path`, opérés par Strimzi 0.45.0.

Sur les deux points :

Un seul broker — cela signifie aucune réplication : le facteur de réplication ne peut pas dépasser 1, donc la perte du broker entraîne la perte des messages non consommés. C’est acceptable pour valider la chaîne fonctionnelle visée par la Release 0, ça ne l’est pas en exploitation. En production, il faut au minimum trois brokers avec un facteur de réplication de 3 et `min.insync.replicas` à 2.

ZooKeeper plutôt que KRaft — KRaft est le mode moderne, qui supprime la dépendance à ZooKeeper. Strimzi 0.45 le supporte, donc ce n’était pas une contrainte technique : c’est le mode par défaut historique qui a été conservé, sans arbitrage explicite. À refaire, KRaft serait le bon choix — moins de composants à opérer, démarrage plus rapide.

Stratégie de défense. Sur ce genre de question, la pire réponse est de justifier après coup un choix qui n’en était pas un. Distingue clairement ce qui était un arbitrage conscient (un broker, faute de ressources sur un cluster à trois nœuds) de ce qui est un défaut d’attention (ZooKeeper par défaut). Le jury respecte cette lucidité.

Q4.5 — [PIÈGE] Ta connexion Kafka est-elle chiffrée ?

Réponse. Non. La connexion AdminClient du backend vers le service de bootstrap ne configure ni TLS ni SASL : l'écouteur emprunté est en clair, sur le port 9092.

Le raisonnement qui rend ça acceptable : ce trafic ne quitte jamais le réseau interne du cluster, il va du namespace `platform` au namespace `kafka`. Mais ce raisonnement a une limite qu'il faut reconnaître — il suppose que le réseau interne du cluster est un périmètre de confiance, ce qui est précisément l'hypothèse que remet en cause une approche zero-trust.

Point aggravant à connaître avant que le jury ne le trouve : **deux écouteurs de type LoadBalancer sont exposés sur le port 9093**, pour permettre la connexion d'outils de développement externes. Ceux-là sortent du cluster. Leur configuration est examinée dans l'audit de sécurité du mémoire.

Ce qu'il faudrait faire. Activer les écouteurs TLS de Strimzi, qui génère lui-même les certificats, et l'authentification SCRAM-SHA-512 ou mTLS pour les clients. Strimzi rend ça déclaratif, c'est quelques lignes dans la ressource `Kafka` — le coût est faible, l'absence tient à une priorisation, pas à une difficulté technique.

5. Multi-tenance, quotas et facturation

Q5.1 — Comment calcules-tu la facture d'un client ?

Réponse. Le modèle est un échantillonnage horaire. Chaque heure, un job planifié parcourt toutes les applications non supprimées et enregistre un instantané de consommation : CPU demandé en vCPU, mémoire en Go, nombre de réplicas, et un facteur de disponibilité.

Le coût d'une heure vaut : $(\text{vCPU} \times 0,048 + \text{Go} \times 0,006) \times \text{réplicas} \times \text{facteur}$.

Le facteur de disponibilité traduit l'état réel de l'application : 1,0 si elle tourne, 0,0 si elle est en échec — un déploiement raté n'est pas facturé — et 0,2 pour les autres états, notamment le scale-to-zero. Ce 0,2 représente le coût résiduel de réservation d'une application enregistrée mais inactive.

La facture mensuelle agrège ensuite ces instantanés par application et par période.

Preuve dans le code. `BillingService` : constantes `CPU_PER_VCPU_HOUR = 0.048`, `MEM_PER_GB_HOUR = 0.006`, `HOURS_MONTH = 720`, méthode `uptimeFactor(App)` avec le `switch` sur le statut, et `takeSnapshot()` qui persiste chaque `BillingSnapshot`.

Planification dans `BillingScheduler` : `@Scheduled(cron = "0 0 * * * *")` pour l'instantané horaire, `"0 0 8 * * *"` pour la vérification quotidienne des impayés, `"0 5 0 1 * *"` pour la génération des factures le 1er du mois.

Relance probable : « pourquoi factures-tu le CPU *demandé* et pas le CPU *consommé* ? ». Réponse : parce que Kubernetes réserve la ressource demandée sur le nœud, qu'elle soit utilisée ou non — elle est indisponible pour les autres tenants. C'est le modèle des

fournisseurs cloud, et il incite le client à dimensionner justement. Facturer la consommation réelle exigerait d'agréger des métriques Prometheus sur toute la période, avec une complexité et une imprécision supérieures.

Q5.2 — Que se passe-t-il pour la facturation quand un client supprime son application ?

Réponse. L'application n'est pas effacée de la base : son statut passe à `DELETED`, et le service Knative est réellement détruit sur le cluster. Ce choix est délibéré et sert la facturation : les instantanés antérieurs sont conservés, donc ce qui était dû avant la suppression reste facturable, mais aucun coût nouveau ne s'accumule puisque `takeSnapshot()` filtre explicitement les applications `DELETED`.

Preuve dans le code. `AppService.deleteApp()` fait `app.setStatus("DELETED")` après `knativeService.delete(...)`, avec le commentaire explicite « billing history must be preserved ». Et dans `BillingService.takeSnapshot()` : `.filter(a -> !"DELETED".equals(a.getStatus()))`.

Pourquoi c'est une bonne réponse. Elle montre que tu as pensé à la cohérence entre deux domaines fonctionnels — un client ne doit pas pouvoir effacer sa dette en supprimant son application, ni être facturé pour une ressource qui n'existe plus.

Q5.3 — Comment empêches-tu un client de consommer toutes les ressources du cluster ?

Réponse. Deux mécanismes complémentaires.

Un quota applicatif d'abord : chaque tenant a un `TenantQuota` avec des valeurs par défaut de 2000m de CPU, 4 Gio de mémoire et 10 applications. Avant toute création d'application, `assertCanCreateApp()` compte les applications actives et lève une `ConflictException` (HTTP 409) si le plafond est atteint — avec un message explicite, pas une erreur 500 opaque.

Un quota Kubernetes ensuite : le quota est synchronisé vers le cluster sous forme de `ResourceQuota` nommée `tenant-quota` dans le namespace du tenant. C'est important, parce que cette seconde barrière est appliquée par Kubernetes lui-même : même si le backend était contourné, le cluster refuserait le dépassement.

Preuve dans le code. `QuotaService` : constantes `DEFAULT_MAX_CPU = "2000m"`, `DEFAULT_MAX_MEMORY = "4Gi"`, `DEFAULT_MAX_APPS = 10` ; méthodes `assertCanCreateApp()` et `syncToCluster(namespace, quota)`.

En complément, un système d'alerte budgétaire prévient le client quand sa consommation mensuelle dépasse un seuil configurable (50 \$ par défaut).

Q5.4 — Un ADMIN peut suspendre un client. Techniquement, ça fait quoi ?

Réponse. La suspension d'une application individuelle patche les annotations d'autoscaling du service Knative pour mettre `maxScale` à zéro : l'application ne peut plus démarrer d'instance, et tout trafic entrant reçoit un 503. Le statut passe à `SUSPENDED` en base.

La suspension d'un client entier applique le même traitement à toutes ses applications et marque l'utilisateur `suspended`. La restauration inverse l'opération en rétablissant les bornes d'origine.

L'intérêt de cette approche par rapport à une suppression : elle est **réversible et non destructive**. Les données du client, ses configurations et ses topics restent intacts. C'est ce qu'on veut pour un impayé ou un abus présumé, où la décision peut être revue.

Preuve dans le code. `AdminController` : `POST /api/admin/apps/{id}/suspend` avec l'opération documentée « Suspend a specific app (scale to zero, 503 on traffic) », `POST /api/admin/clients/{userId}/suspend` qui itère sur les applications, et `KnativeService.scaleService(serviceName, namespace, minReplicas, maxReplicas)`.

Q5.5 — Qu'est-ce qui est tracé dans ton journal d'audit ?

Réponse. Uniquement les actions de gouvernance de l'ADMIN, jamais les lectures. Huit types exactement : `SUSPEND_CLIENT`, `RESTORE_CLIENT`, `SUSPEND_APP`, `RESTORE_APP`, `FORCE_DELETE_APP`, `FORCE_DELETE_TOPIC`, `UPDATE_QUOTA`, `SCALE_APP`.

Chaque entrée enregistre l'acteur, l'action, la cible, l'état avant, l'état après et l'adresse IP source. Le journal est consultable et exportable par l'ADMIN.

Le choix à défendre. Ne tracer que les actions modifiantes est délibéré : un journal qui enregistrerait chaque consultation deviendrait volumineux et illisible, noyant les événements qui comptent réellement. L'audit répond ici à la question « qui a changé quoi, et quel était l'état avant ? », qui est celle d'un litige ou d'un incident.

Preuve dans le code. L'énumération `AdminAction`, et les appels `auditLogService.record(actorId, actorName, AdminAction.X, "APP", id, avant, après, null, clientIp(request))` dans chaque méthode modifiante de `AdminController`.

6. Développement backend et conception

Q6.1 — Ton déploiement est asynchrone. Pourquoi, et comment ?

Réponse. Parce que la création d'un service Knative n'est pas instantanée : il faut tirer l'image, démarrer le conteneur, attendre que la Revision devienne Ready. Cela peut prendre des dizaines de secondes. Bloquer la requête HTTP pendant ce temps donnerait une interface figée et risquerait un timeout.

Le backend répond donc immédiatement `201 Created` avec le statut `DEPLOYING`, puis déclenche le déploiement réel en tâche de fond. L'interface est notifiée de la progression par un flux SSE, et le statut converge vers `RUNNING` ou `FAILED`.

Preuve dans le code — et c'est le meilleur détail technique de tout ton projet.

`AppDeploymentAsyncRunner` est un bean **séparé** de `AppService`, et la classe porte un commentaire qui explique pourquoi :

@Async ne prend effet que sur les appels qui passent par le proxy Spring du bean où il est déclaré. Un appel intra-classe (AppService appelant sa propre méthode @Async) contourne entièrement ce proxy et s'exécute de façon synchrone.

C'est un piège classique de Spring, et l'avoir identifié et contourné explicitement est un vrai argument de maîtrise. Si tu ne dois retenir qu'une seule preuve de code pour ta soutenance, c'est celle-là.

Q6.2 — Comment le statut d'une application reste-t-il à jour ?

Réponse. Par deux mécanismes complémentaires, un passif et un actif.

Le mécanisme actif est `KnativeWatcher` : au démarrage, il ouvre un watch permanent sur les ressources Knative de tous les namespaces. À chaque événement du cluster, il met à jour le statut en base. Il gère sa propre reconnexion en cas de coupure.

Le mécanisme passif est `syncStatusFromKubernetes()` : à chaque consultation de la liste ou du détail d'une application, le backend interroge le cluster pour connaître le statut réel et le compare à celui en base. La correspondance est explicite — `RUNNING` reste `RUNNING`, `SCALED_TO_ZERO` devient `IDLE`, `NOT_FOUND` devient `DELETED`. La base n'est réécrite que si quelque chose a effectivement changé, pour éviter des écritures inutiles.

Pourquoi les deux ? Le watcher couvre les changements survenant quand personne ne regarde. La synchronisation à la lecture couvre le cas où le watcher aurait manqué un événement, par exemple après un redémarrage du backend. C'est une redondance volontaire, pas une duplication accidentelle.

Q6.3 — Comment détectes-tu qu'une application du client plante en boucle ?

Réponse. Un job planifié toutes les cinq minutes interroge le cluster à la recherche de pods dont le compteur de redémarrages dépasse cinq — la signature d'un `CrashLoopBackOff`. Pour chaque application concernée, une alerte est créée dans les journaux de déploiement avec le type `CRASH_LOOP_ALERT` et poussée en temps réel vers l'interface du client.

Un mécanisme anti-spam évite de renvoyer la même alerte : si une alerte du même type existe déjà pour cette application depuis moins d'une heure, elle n'est pas dupliquée.

Preuve dans le code. `CrashLoopScheduler` avec `@Scheduled(fixedRate = 5 * 60 * 1000)`, et `AppService.checkCrashLoops()` avec les constantes `CRASH_LOOP_RESTART_THRESHOLD = 5` et `CRASH_LOOP_ALERT_COOLDOWN_HOURS = 1`.

Ce que ça montre. Que tu as pensé à l'expérience d'exploitation du client, pas seulement au chemin nominal. C'est le genre de fonctionnalité qui distingue une plateforme d'une démo.

Q6.4 — Comment fonctionne le flux de journaux en temps réel ?

Réponse. Par Server-Sent Events (SSE), un canal HTTP unidirectionnel du serveur vers le navigateur. C'est le choix adapté ici : contrairement à WebSocket, on n'a pas besoin de communication bidirectionnelle, et SSE gère nativement la reconnexion automatique.

Deux flux distincts existent : les journaux de déploiement produits par la plateforme, et les journaux bruts du conteneur, récupérés directement depuis l'API Kubernetes.

Point de sécurité à connaître. SSE via l'objet `EventSource` du navigateur ne permet pas d'envoyer d'en-tête `Authorization` personnalisé. La première implémentation passait donc le jeton **dans les paramètres d'URL** — ce qui le rend visible dans les journaux d'accès du serveur, l'historique du navigateur et les en-têtes `Referer`. Cette vulnérabilité a été identifiée et corrigée ; un filtre dédié, `SseTokenFilter`, gère désormais l'authentification de ces flux.

Pourquoi le citer. C'est un correctif de sécurité réel, trouvé et traité pendant le projet. Le mentionner spontanément est bien plus valorisant que de le laisser découvrir.

Q6.5 — [PIÈGE] Pourquoi as-tu une dépendance Elasticsearch qui ne sert à rien ?

Réponse. C'est exact et je l'ai documenté. `spring-boot-starter-data-elasticsearch` figure dans les dépendances du backend, mais **aucune classe du code source ne l'utilise** : pas de dépôt Elasticsearch, pas d'entité indexée, aucun appel client.

Le chemin réel des journaux est direct : API Kubernetes → service de journaux → diffusion SSE → portail client.

C'est un reliquat d'une intention initiale — indexer les journaux pour permettre la recherche plein texte — qui n'a pas été menée à terme, la priorité étant allée au temps réel plutôt qu'à la recherche historique. La dépendance aurait dû être retirée.

Stratégie de défense. Ne cherche pas à faire croire qu'Elasticsearch est utilisé « en partie ». Une dépendance morte est un défaut d'hygiène, pas une faute grave. Ce qui compte, c'est que tu l'aies détectée par une analyse de ton propre code et documentée honnêtement dans le mémoire, plutôt que de laisser croire à une architecture qui n'existe pas.

7. DevOps et industrialisation

Q7.1 — Comment déploies-tu la plateforme elle-même ?

Réponse. Par des pipelines Jenkins, un par composant : backend, portail client et console d'administration. Chaque pipeline construit l'image Docker, la pousse sur le registre, et met à jour le déploiement sur le cluster.

Une distinction importante à faire : les pipelines couvrent les **composants applicatifs** de la plateforme. L'infrastructure elle-même — le cluster, Knative Serving, Strimzi, MetalLB — a été provisionnée directement sur les machines et **n'est pas versionnée** sous forme de manifestes dans le dépôt.

Limite à reconnaître. C'est le principal manque d'industrialisation du projet. Une reconstruction complète du cluster depuis zéro dépendrait de la mémoire de l'opérateur plutôt que d'un dépôt. La solution est identifiée : décrire l'infrastructure en Infrastructure as Code (Terraform pour les machines, Helm ou Kustomize pour les composants Kubernetes), voire adopter une approche GitOps avec ArgoCD où l'état du cluster est en permanence réconcilié avec le dépôt Git.

Q7.2 — As-tu des tests automatisés dans tes pipelines ?

Réponse. Non, et c'est identifié comme une limite dans le chapitre de validation. La validation a été conduite manuellement, endpoint par endpoint, avec Postman et par vérification directe sur le cluster.

Ce qu'il faudrait, par ordre de priorité. Des tests unitaires d'abord sur la logique métier pure et testable sans infrastructure : `BillingService.parseVcpu()`, `parseGb()`, `uptimeFactor()` et `hourlyRate()` sont des fonctions statiques déterministes, ce sont les candidats les plus évidents. Ensuite des tests d'intégration avec Testcontainers pour PostgreSQL et Kafka. Enfin des tests de bout en bout sur un cluster éphémère de type kind.

Stratégie de défense. N'essaie pas de faire passer les tests manuels pour une stratégie de test. Assume : les tests automatisés ont été arbitrés en défaveur des fonctionnalités, sur un projet à périmètre large et à durée contrainte. Ce qui sauve la réponse, c'est de montrer que tu sais exactement **par où commencer** si on te donnait une semaine de plus — cite `BillingService`, c'est concret et ça prouve que tu as réfléchi au problème.

Q7.3 — Comment le backend s'authentifie-t-il auprès de l'API Kubernetes ?

Réponse. Par le mécanisme natif de Kubernetes : un ServiceAccount monté dans le pod du backend, associé à un ClusterRole via un ClusterRoleBinding. Le client Fabric8

détecte automatiquement ce jeton et le certificat de l'autorité de certification depuis le système de fichiers du pod.

Ce ClusterRole a fait l'objet d'un travail de réduction : il a été reconstruit pour se limiter aux permissions réellement nécessaires, dans une démarche de moindre privilège. Il a ensuite dû être étendu à la lecture des nœuds et des événements, pour permettre à la console d'administration d'afficher la supervision du cluster.

Point de vigilance. Il s'agit d'un **ClusterRole**, pas d'un Role limité à un namespace, parce que le backend doit agir dans tous les namespaces des tenants — il en crée de nouveaux à la volée. C'est structurellement nécessaire, mais cela signifie qu'une compromission du backend donnerait un accès étendu au cluster. C'est le point de sécurité le plus sensible de l'architecture, et il faut le formuler comme tel : le backend est le point d'entrée unique vers le cluster, ce qui centralise les contrôles mais concentre aussi le risque.

8. Questions de synthèse et de recul

Q8.1 — Si tu devais refaire ce projet, que changerais-tu en priorité ?

Réponse structurée en trois points, du plus important au moins important :

1. **Le flow d'authentification.** Authorization Code + PKCE dès le départ, au lieu de ROPC. C'est le seul choix qui touche à la sécurité du produit lui-même et qui conditionne des évolutions futures (SSO, MFA, fédération).
2. **L'Infrastructure as Code.** Décrire le cluster et ses composants dans le dépôt dès le Sprint 0. Le coût initial est réel, mais l'absence de description rend la plateforme non reproductible — c'est le manque qui a le plus d'impact sur l'exploitabilité.
3. **Les tests automatisés.** Au minimum sur la logique de facturation, qui est du calcul pur, sans dépendance, et dont une erreur aurait une conséquence directe et visible pour le client.

Pourquoi cette question est un cadeau. Elle teste ta capacité de recul, pas ta mémoire. Un candidat qui répond « rien, tout était bien » perd immédiatement en crédibilité. Un candidat qui hiérarchise ses regrets montre qu'il sait arbitrer — c'est exactement la compétence attendue d'un ingénieur.

Q8.2 — Ta plateforme tiendrait combien de clients en l'état ?

Réponse. En l'état, très peu, et il faut être précis sur ce qui limite. Le facteur bloquant n'est pas le code applicatif mais le dimensionnement de l'infrastructure : trois nœuds, un seul broker Kafka, un volume de 1 Gio, une instance unique du backend et de PostgreSQL.

Les points de rupture, dans l'ordre où ils se manifesteraient :

- **PostgreSQL en instance unique** : point de défaillance unique, aucune réplication.
- **Le backend en instance unique** : `KnativeWatcher` et les jobs planifiés (`@Scheduled`) s'exécuteraient en double si on répliquait le backend naïvement — il faudrait un mécanisme de verrou distribué type `ShedLock` avant de pouvoir passer à l'échelle horizontalement. C'est un point subtil et précis, qui montre que tu as réfléchi au problème.
- **Kafka à un broker** : aucune tolérance de panne.
- **La capacité des trois nœuds** : plafond mécanique sur le nombre de pods.

Ce qui, en revanche, est prêt à passer à l'échelle. Le modèle multi-tenant lui-même : l'isolation par namespace, la résolution du contexte utilisateur et les quotas ne font aucune hypothèse sur le nombre de tenants. C'est l'infrastructure qu'il faudrait renforcer, pas l'architecture logicielle à repenser. C'est une distinction importante à faire valoir.

Q8.3 — Quelle a été la plus grande difficulté technique du projet ?

Réponse (choisis-en une et raconte-la vraiment). La plus défendable est l'intégration `Knative Eventing`, parce qu'elle combine plusieurs difficultés réelles :

- Il n'existe **pas de client Java officiel** pour les ressources `Knative`. Il a fallu passer par l'API générique de `Fabric8`, en construisant les manifestes comme des structures de données non typées — plus verbeux et sans vérification à la compilation.
- La contrainte de colocalisation `Broker/Trigger` dans le même namespace n'est pas évidente à la lecture de la documentation, et se manifeste par un `Trigger` qui reste silencieusement non fonctionnel. D'où `ensureBrokerExists()` appelé avant toute création.
- Le chaînage `KafkaSource` → `Broker` → `Trigger` → `Service` comporte plusieurs points de rupture possibles, et un maillon défaillant ne produit pas d'erreur franche : l'événement disparaît simplement.

Comment bien raconter ça. Structure en trois temps : le symptôme observé, la démarche de diagnostic, la correction. C'est le format attendu, et c'est ce qui distingue une difficulté réellement vécue d'une difficulté récitée.

Q8.4 — Ton projet, c'est un PaaS, un FaaS ou un CaaS ?

Réponse. C'est un **PaaS à exécution serverless**, et la nuance mérite d'être posée.

Ce n'est pas un FaaS au sens strict : le client déploie une **image Docker complète**, une application autonome avec son runtime et son serveur HTTP, pas une fonction isolée que la plateforme exécuterait dans un runtime qu'elle fournit. Il n'y a ni catalogue de langages, ni packaging de fonction.

Ce n'est pas non plus un simple CaaS : le client ne manipule ni Deployment, ni Service, ni Ingress, ni réplicas. Il fournit une image, un port et des bornes de mise à l'échelle. Tout le reste — namespace, isolation réseau, routage, URL, autoscaling, facturation — est pris en charge par la plateforme.

C'est donc bien un PaaS, dont la particularité est le modèle d'exécution serverless hérité de Knative : scale-to-zero et facturation à l'usage réel.

Pourquoi cette question compte. Elle vérifie que tu maîtrises le vocabulaire de ton domaine. Une confusion entre ces trois modèles suggérerait que tu as construit sans bien situer ce que tu construisais.

Aide-mémoire — chiffres et noms à connaître par cœur

Élément	Valeur
Nœuds du cluster	3 (1 control-plane, 2 workers)
Version Kubernetes	v1.30.3
Système / runtime	Ubuntu 24.04.4 LTS, containerd 2.2.1
CNI	Cilium 1.19.1
Strimzi	0.45.0
Rôles	ADMIN, CLIENT_ADMIN, MEMBER
Permissions granulaires	8
Actions auditées	8
Quotas par défaut	2000m CPU, 4 Gio, 10 applications
Tarif CPU	0,048 \$ / vCPU / heure
Tarif mémoire	0,006 \$ / Go / heure
Facteur de disponibilité	1,0 RUNNING · 0,0 FAILED · 0,2 autres
Seuil d'alerte budget	50 \$ / mois (configurable)
Seuil crash-loop	5 redémarrages, alerte max 1×/heure
Défauts d'application	port 8080, 100m CPU, 128Mi RAM, minScale 0, maxScale 10

Les cinq questions à préparer en priorité

Si le temps manque, concentre-toi sur celles-ci — ce sont les plus probables et les plus discriminantes.

#	Question	Sujet	Pourquoi celle-ci
1	Q3.2	Le flow OAuth2 (ROPC)	La plus dangereuse. Un jury qui connaît OAuth2 la posera.
2	Q2.1	L'isolation multi-tenant en trois couches	Le cœur du projet.
3	Q4.2	Le parcours d'un événement Kafka jusqu'au réveil de l'application	La question technique la plus riche.
4	Q6.1	Le déploiement asynchrone et le piège du proxy Spring	Ta meilleure preuve de maîtrise.
5	Q8.1	Ce que tu changerais	La question de recul, presque toujours posée.

Toutes les affirmations techniques de ce document sont vérifiées dans le code source du projet. Aucune donnée, mesure ou fonctionnalité n'a été inventée.